

7图

1

1用“边集”的文字形式表示该图；

$E(G)=\{(A,C),(B,C),(C,D),(C,E),(D,F),(E,F)\}$

2. 请写出拓扑排序的伪代码，并写出一个合法的拓扑序列；

函数 `topologicalSort(图 G)`:

初始化入度映射 `inDegree`，对于所有节点，设置 `inDegree[node] = 0`

对于每条边 (u, v) 在 G 中:

`inDegree[v] += 1`

初始化队列 Q ，将所有满足 `inDegree[node] == 0` 的节点加入 Q

初始化空列表 L 用于存储拓扑序列

当 Q 不为空:

从 Q 中取出节点 u

将 u 加入 L

对于 u 的每个邻居 v :

`inDegree[v] -= 1`

如果 `inDegree[v] == 0`:

将 v 加入 Q

如果 L 的长度等于节点数量:

返回 L

否则:

返回 "图中有环"

一个合法的拓扑排序序列是A、B、C、D、E、F

3. 计算完成所有任务的最短总时间（关键路径长度）

根据任务持续时间和依赖关系，使用关键路径方法计算：

关键路径是任务 $B \rightarrow C \rightarrow E \rightarrow F$ 的路径时间为 $3 + 4 + 3 + 1 = 11$

其他路径时间均大于或等于 11

因此，完成所有任务的最短总时间（关键路径长度）为 11。

2

伪代码

函数 Dijkstra(图 G, 源点 source):

 初始化距离数组 **dist**, 对于每个顶点 **v**:

dist[v] $\leftarrow$ 无穷大

prev[v] $\leftarrow$ undefined // 用于记录路径的前驱节点

dist[source] $\leftarrow$ 0

 初始化优先队列 **Q**, 包含所有顶点, 按 **dist** 值排序

 当 **Q** 不为空:

u $\leftarrow$ 从 **Q** 中取出具有最小 **dist** 的顶点

 对于 **u** 的每个邻居 **v**:

alt $\leftarrow$ **dist[u]** + weight(**u**, **v**)

 如果 **alt** < **dist[v]**:

dist[v] $\leftarrow$ **alt**

prev[v] $\leftarrow$ **u**

 更新 **Q** 中 **v** 的优先级

 返回 **dist**

- 初始化：设置源点 A 的距离为 0，即 $\text{dist}[A] = 0$ 。其他顶点的距离初始化为无穷大： $\text{dist}[B] = \infty$, $\text{dist}[C] = \infty$, $\text{dist}[D] = \infty$, $\text{dist}[E] = \infty$ 。所有顶点均未处理。
- 处理顶点 A：
 检查 A 的邻居 B 和 C：
 A $\rightarrow$ B: $\text{alt} = \text{dist}[A] + 10 = 0 + 10 = 10$ ，小于当前 $\text{dist}[B] = \infty$ ，更新 $\text{dist}[B] = 10$ 。
 A $\rightarrow$ C: $\text{alt} = \text{dist}[A] + 3 = 0 + 3 = 3$ ，小于当前 $\text{dist}[C] = \infty$ ，更新 $\text{dist}[C] = 3$ 。
 当前距离： $\text{dist}[A] = 0$, $\text{dist}[B] = 10$, $\text{dist}[C] = 3$, $\text{dist}[D] = \infty$, $\text{dist}[E] = \infty$ 。
 标记 A 为已处理。
- 处理顶点 C（未处理顶点中距离最小）：
 检查 C 的邻居 B、D、E：
 C $\rightarrow$ B: $\text{alt} = \text{dist}[C] + 4 = 3 + 4 = 7$ ，小于当前 $\text{dist}[B] = 10$ ，更新 $\text{dist}[B] = 7$ 。
 C $\rightarrow$ D: $\text{alt} = \text{dist}[C] + 8 = 3 + 8 = 11$ ，小于当前 $\text{dist}[D] = \infty$ ，更新 $\text{dist}[D] = 11$ 。
 C $\rightarrow$ E: $\text{alt} = \text{dist}[C] + 2 = 3 + 2 = 5$ ，小于当前 $\text{dist}[E] = \infty$ ，更新 $\text{dist}[E] = 5$ 。

当前距离：dist[A] = 0, dist[B] = 7, dist[C] = 3, dist[D] = 11, dist[E] = 5。

标记 C 为已处理。

- 处理顶点 E（未处理顶点中距离最小）：

检查 E 的邻居 D：

$E \rightarrow D: \text{alt} = \text{dist}[E] + 9 = 5 + 9 = 14$ ，大于当前 $\text{dist}[D] = 11$ ，不更新。

当前距离不变：dist[A] = 0, dist[B] = 7, dist[C] = 3, dist[D] = 11, dist[E] = 5。

标记 E 为已处理。

处理顶点 B（未处理顶点中距离最小）：

检查 B 的邻居 C 和 D：

$B \rightarrow C: \text{alt} = \text{dist}[B] + 1 = 7 + 1 = 8$ ，大于当前 $\text{dist}[C] = 3$ ，不更新。

$B \rightarrow D: \text{alt} = \text{dist}[B] + 2 = 7 + 2 = 9$ ，小于当前 $\text{dist}[D] = 11$ ，更新 $\text{dist}[D] = 9$ 。

当前距离：dist[A] = 0, dist[B] = 7, dist[C] = 3, dist[D] = 9, dist[E] = 5。

标记 B 为已处理。

- 处理顶点 D（最后一个未处理顶点）：

检查 D 的邻居 E：

$D \rightarrow E: \text{alt} = \text{dist}[D] + 7 = 9 + 7 = 16$ ，大于当前 $\text{dist}[E] = 5$ ，不更新。

当前距离不变：dist[A] = 0, dist[B] = 7, dist[C] = 3, dist[D] = 9, dist[E] = 5。

标记 D 为已处理。

- 最终从 A 到各顶点的最短路径长度为：

A → A: 0

A → B: 7

A → C: 3

A → D: 9

A → E: 5

3

1. Kruskal 算法求最小生成树

使用 Kruskal 算法，按权值从小到大选择边，确保不形成环。步骤如下：

排序所有边：D-E (1), D-F (2), B-C (2), A-C (3), C-D (3), E-F (3), A-B (4), C-E (4), B-D (5)

初始化空 MST。

选入 D-E (权值 1), MST 边集: {D-E}

选入 D-F (权值 2), MST 边集: {D-E, D-F}

选入 B-C (权值 2), MST 边集: {D-E, D-F, B-C}

选入 A-C (权值 3), MST 边集: {D-E, D-F, B-C, A-C}

选入 C-D (权值 3), MST 边集: {D-E, D-F, B-C, A-C, C-D} (此时 MST 包含 5 条边, 连接所有 6 个顶点, 算法停止)

因此, 每次选入的边顺序为: D-E, D-F, B-C, A-C, C-D。

2. 最小生成树的总权值

MST 边权值: D-E:1, D-F:2, B-C:2, A-C:3, C-D:3。总权值 = $1 + 2 + 2 + 3 + 3 = 11$ 。

3. 次小生成树 (SMST) 的求解思路和伪代码

求解思路

次小生成树是总权值严格大于最小生成树 (MST) 但最小的生成树。常用方法如下:

1. 首先找到 MST T 。
2. 对于每条不在 T 中的边 e , 将其加入 T , 这会形成一个环。
3. 在环中找到权值最大的边 f (不包括 e 本身), 移除 f , 得到新生成树 $T' = T \cup \{e\} \setminus \{f\}$ 。
4. 计算 T' 的权值 $w(T')$ 。
5. 比较所有这样的 T' , 取 $w(T') > w(T)$ 且最小的那个作为次小生成树。

伪代码

函数 SMST(G):

```
// G 为图，包含顶点集 V 和边集 E
T ← Kruskal(G) // 获取最小生成树
smst_value ← ∞
smst_tree ← null

// 构建树 T 的邻接表表示，用于路径查询
构建邻接表 adj 对于 T

// 对于每条不在 T 中的边 e
for each edge e = (u, v) in E \ T:
    // 在 T 中查找从 u 到 v 的路径 P
    P ← findPath(adj, u, v) // 返回路径上的边集合

    // 找到路径 P 中权值最大的边 f
    max_weight ← -∞
    for each edge f in P:
        if weight(f) > max_weight:
            max_weight ← weight(f)
            max_edge ← f

    // 计算新树 T' 的权值
    w_new ← weight(T) - weight(max_edge) + weight(e)

    // 更新次小生成树
    if w_new > weight(T) and w_new < smst_value:
        smst_value ← w_new
        smst_tree ← T ∪ {e} \ {max_edge}

return smst_value, smst_tree
```

// 辅助函数：在树 T 中查找从 u 到 v 的路径（基于 BFS）

函数 findPath(adj, u, v):

```
初始化 visited 数组为 false
初始化 parent 映射记录每个节点的父节点和边
队列 Q ← 空队列
Q.enqueue(u)
visited[u] ← true
while Q 不为空:
    current ← Q.dequeue()
    if current == v:
```

```

        break
    for each neighbor w in adj[current]:
        if not visited[w]:
            visited[w] ← true
            parent[w] ← (current, edge(current, w)) // 记录边
            Q.enqueue(w)

// 回溯路径
path_edges ← []
node ← v
while node != u:
    (prev, edge) ← parent[node]
    path_edges.append(edge)
    node ← prev
return path_edges

```

对本图

- MST T 权值为 11。
- 不在 T 中的边：A-B (4), B-D (5), C-E (4), E-F (3)。
- 计算各 T' 权值：
 - 加入 E-F，移除 D-F，权值 12。
 - 加入 A-B，移除 A-C，权值 12。
 - 加入 B-D，移除 C-D，权值 13。
 - 加入 C-E，移除 C-D，权值 12。
- 次小生成树权值为 12。

因此，次小生成树的总权值为 12。